Obchodní akademie, Vyšší odborná škola a Jazyková škola

s právem státní jazykové zkoušky Uherské Hradiště

MATURITNÍ PRÁCE

ACHIEVO – WEBOVÁ APLIKACE

2026

Rygulski Martin, 4.B

Vnitřní předpis OA, VOŠ a JŠ, čj. 025/ORG/2023

Příloha č. 3

ZADÁNÍ MATURITNÍ PRÁCE INFORMAČNÍ TECHNOLOGIE

Jméno žáka:	Martin Rygulski
Téma maturitní práce:	Achievo – webová aplikace
Vedoucí práce:	Mgr. Karel Jindra
Oponent práce:	Ing. Martin Šimůnek
Způsob zpracování:	Žák navrhne a implementuje webovou aplikaci s názvem **Achievo**, která pomůže studentům efektivně organizovat čas, školní povinnosti a úkoly. Aplikace bude navržena tak, aby spojovala prvky to-do listu a kalendáře, doplněné o gamifikační mechanismy, které mají uživatele motivovat k pravidelnému plnění povinností. Díky tomu nebude Achievo pouze klasickým plánovačem, ale i nástrojem, který podporuje dlouhodobou disciplínu a motivaci. Aplikace bude splňovat několik základních požadavků. Uživatel se bude schopen zaregistrovat a přihlásit do systému, následně si vytvářet a spravovat úkoly, přiřazovat jim termíny a označovat je jako splněné. Dále bude součástí implementace kalendář, ve kterém se budou zobrazovat všechny události, například testy, školní projekty či rozvrh hodin. Významnou součástí bude také gamifikace – za plnění úkolů uživatel získává body a postupuje na vyšší úrovně. Systém rovněž bude poskytovat statistiky, které budou ukazovat, kolik úkolů bylo splněno včas a jak se vyvíjí produktivita uživatele. Kromě funkčních požadavků budou kladeny také nároky na přehledné uživatelské rozhraní, bezpečné ukládání hesel a bezproblémový chod na běžném webhostingu s podporou PHP a MySQL. Minimálním produktem, který tyto požadavky naplňuje, je aplikace umožňující registraci a přihlášení uživatele, vytváření úkolů s termínem, jejich označení jako splněných a základní kalendářové zobrazení. Tento základ lze dále rozšiřovat o bodový systém, statistiky a notifikace. Pokud by minimální varianta působila příliš jednoduše, je vhodné doplnit alespoň základní gamifikační prvky, aby aplikace získala na atraktivitě a odlišila se od běžných řešení.

	Primárními uživateli jsou studenti středních a vysokých škol, kteří potřebují mít přehled o svých povinnostech, projektech či testech. Aplikace bude navržena jako webová, dostupná prostřednictvím běžného internetového prohlížeče. Serverová část bude realizována v jazyce PHP s databází MySQL či MariaDB a frontend bude využívat HTML5, CSS (například s frameworkem Bootstrap) a JavaScript. Díky tomu bude aplikace snadno přístupná z počítače i mobilního telefonu. Cílovou věkovou skupinou jsou studenti ve věku přibližně 15 až 25 let, kteří zvládají základní práci s počítačem a internetem, avšak nemusí mít žádné technické znalosti ani zkušenosti s programováním. V dokumentaci žák srovná svou aplikaci s exitujícími produkty jako jsou Google Kalendář, Todoist nebo Microsoft To Do. Aplikace bude otestována minimálně třemi testery a poznatky z testování budou uvedeny v dokumentaci.
Pokyny k průběhu vypracování:	Práci bude žák vést v repositáři na GitHubu či obdobném včetně historie práce od zadání až po odevzdání projektu. Zdroje informací žák uvede dle normy ČSN ISO 690. Pokud je zdrojem informací AI, žák je cituje dle vnitřní směrnice čj. 025/ORG/2023 školy.
Pokyny k odevzdání:	**Žák odevzdá práci v tištěné a elektronické podobě** • Tištěná podoba práce obsahuje uživatelskou a technickou dokumentaci. **Tištěnou podobu** (v kroužkové vazbě) žák odevzdá na studijní oddělení školy. **Elektronická podoba práce obsahuje** • Dokumentaci ve formátu PDF/A • Resumé ve formátu PDF/A • Výsledný projekt, zdrojové soubory a potřebné knihovny pro spuštění projektu • Prezentaci projektu **Elektronická podoba** práce se nahrává do IS školy dle pokynů vedoucího práce nebo vedení školy. V případě, že se jedná o projekt, na kterém pracovalo více žáků, je povinnou součástí dokumentace podrobné rozdělení činností při práci na projektu.
Kritéria hodnocení:	Hodnocení se skládá z celkové kvality zpracování práce, dokumentace, z kvality prezentace při obhajobě práce, diskuse a z průběžného hodnocení žáka v rámci kontrolních dnů.
Obhajoba projektu	Obhajoba projektu se skládá ze dvou části – prezentace projektu (včetně podpůrné elektronické prezentace) a diskuse nad řešením. Celková délka obhajoby je 20 minut, délka prezentace projektu by neměla překročit 10 minut.

30 -09- 2025

.....

Datum

.....

Podpis ředitele školy

Obchodní akademie, Vyšší odborná škola
a Jazyková škola s právem státní
jazykové zkoušky Uherské Hradiště
Nádražní 22
686 01 Uherské Hradiště

Prohlášení:

Souhlasím s tím, že s výsledky mé práce může být naloženo podle uvážení vedoucího maturitní práce a ředitele školy. V případě publikace budu uveden jako spoluautor. Prohlašuji, že jsem na celé maturitní práci pracoval samostatně a veškeré použité zdroje jsem citoval.

V Uherském Hradišti, dne **31.3.2026**

.....

podpis absolventa

RESUMÉ

Tato maturitní práce se zabývá návrhem a realizací webové aplikace Achievo, jejímž cílem je pomoci studentům s organizací času a plánováním školních povinností. Studium je spojeno s velkým množstvím úkolů, projektů a testů, jejichž přehledné evidování bývá pro mnoho studentů problematické. Častým důvodem není nedostatek snahy, ale absence jednoduchého a přehledného systému, který by umožnil efektivní plánování a kontrolu termínů. Na základě této potřeby byl navržen vlastní nástroj, který kombinuje správu úkolů s kalendářovým přehledem a motivačními prvky.

Obsah

RESUMÉ **6**

ÚVOD **9**

CÍL PRÁCE **10**

1 ANALÝZA PRÁCE **11**

1.1 Analýza zadání 11

1.2 Cílová skupina 11

1.3 Analýza konkurence 12

2 NÁVRH PRÁCE **13**

2.1 Návrh databáze 13

2.1.1 Tabulka users 13

2.1.2 Tabulka events 13

2.1.3 Tabulka tasks 13

2.1.4 Tabulka points 14

2.1.5 Vztahy mezi tabulkami 14

2.2 Návrh architektury systému 15

2.2.1 Celková struktura systému 15

2.2.2 Frontend 15

2.2.3 Backend 16

2.2.4 Databázová vrstva 16

2.2.5 Komunikace mezi vrstvami 16

2.2.6 Řízení přístupu 17

2.3 Návrh uživatelského rozhraní 17

2.4 Bezpečnostní návrh 18

3 TECHNICKÉ ZPRACOVÁNÍ **19**

3.1 Použité technologie 19

3.1.1 Jazyk PHP 19

3.1.2 Framework CodeIgniter4 19

3.1.3 Databázoví systém MySQL 19

4 IMPLEMENTACE DŮLEŽITÝCH FUNKCÍ **20**

4.1 Autentizace uživatelů 20

4.1.1 Správa session 21

4.2 Správa úkolů 21

4.2.1 Mazání úkolů 23

4.2.2 Označení úkolu jako splněný ... 23
4.3 Kalendář ... 24
4.4 Gamifikace ... 25
4.5 Administrace ... 26
4.6 Napojení na databázi ... 26
5 UŽIVATELSKÁ DOKUMENTACE ... 28
5.1 Přihlášení do aplikace ... 28
5.2 Orientace v aplikaci ... 29
5.3 Práce s úkoly ... 29
5.3.1 Vytvoření úkolu (učitel, admin) ... 30
5.3.2 Hodnocení úkolů (učitel, admin) ... 30
5.4 Kalendář ... 31
5.5 Dashboard ... 31
5.6 Leaderboard ... 32
5.7 Administrace uživatelů (admin) ... 32
5.8 Odhlášení ... 33
6 UŽIVATELSKÉ TESTOVÁNÍ ... 34
6.1 Metodika testování ... 34
6.2 Scénář testování ... 34
6.3 Jednotlivé testování: ... 34
6.3.1 Tester 1: ... 34
6.3.2 Tester 2: ... 35
6.3.3 Tester 3: ... 35
6.3.4 Tester 4: ... 36
6.3.5 Tester 5: ... 36
6.4 Finální hodnocení pomocí formuláře ... 37
7 PŘÍNOSY PRÁCE ... 38
ZÁVĚR ... 39
SEZNAM POUŽITÉ LITERATURY ... 40
SEZNAM POUŽITÝCH SYMBOLŮ A ZKRATEK ... 42
SEZNAM OBRÁZKŮ ... 43

ÚVOD

Úvod této maturitní práce se zaměřuje na návrh a tvorbu webové aplikace Achievo, jejímž hlavním cílem je pomoci studentům s organizací času a plánováním školních povinností. Dnešní studium přináší velké množství úkolů, projektů, testů a dalších aktivit, které je často složité udržet v hlavě nebo zapisovat různě po papírech a poznámkách. Mnoho studentů tak zbytečně zapomíná na termíny, nestíhá povinnosti nebo nemá jasný přehled o tom, co je potřeba splnit jako první. Problémem většinou není nedostatek snahy, ale spíše chybějící nástroj, který by vše přehledně ukládal a pomohl s plánováním. Existují sice různé digitální aplikace, avšak ne všechny jsou jednoduché, přehledné nebo přizpůsobené přímo studentům. Z tohoto důvodu vznikl nápad vytvořit vlastní aplikaci Achievo, která má studentům nabídnout snadný způsob, jak si organizovat svůj studijní čas. Aplikace kombinuje klasický seznam úkolů s kalendářem, aby měl uživatel neustále přehled o důležitých termínech. Součástí návrhu jsou také motivační prvky, protože se ukazuje, že studenti lépe pracují s nástroji, které je pozitivně podporují. Body, úrovně nebo statistiky mohou pomoci tomu, aby se aplikace používala dlouhodobě a nejen na pár dní. Achievo je tedy nejen pomocníkem při organizaci, ale také nástrojem, který podporuje pravidelnost a zodpovědný přístup ke školním povinnostem. Důležitou součástí projektu je technické zpracování celého systému. Aplikace je vytvořena jako webová, aby byla přístupná z počítače i telefonu bez nutnosti instalace. Funkčnost webové aplikace zajišťuje jazyk PHP a framework Codeigniter. Ten poskytuje přehlednou strukturu a usnadňuje práci s databází. Pro ukládání dat, jako jsou uživatelské účty nebo jednotlivé úkoly, je použita databáze MySQL. Zvláštní důraz je kladen na bezpečnost, zejména na správné a bezpečné ukládání hesel a ochranu osobních údajů. Achievo nevzniká jen jako školní práce, ale také jako projekt, který může být do budoucna rozšířen o další funkce, například upozornění na blížící se termíny, sdílení úkolů mezi uživateli nebo lepší statistiky produktivity. Úvod této práce proto vysvětluje nejen samotný vznik aplikace, ale také její smysl, přínos a důvody, proč tento nástroj může být pro studenty užitečný.

CÍL PRÁCE

Cílem této maturitní práce je navrhnout, vytvořit a otestovat webovou aplikaci Achievo, která studentům usnadní správu času a plánování školních povinností. Hlavním úkolem je vytvořit jednoduché a přehledné prostředí, kde budou moci uživatelé zapisovat své úkoly, pracovat s termíny a organizovat si studium tak, aby měli vždy jasný přehled o svých povinnostech. Součástí cíle je i zajištění toho, aby byla aplikace pro studenty snadno použitelná a motivovala je k tomu, aby ji používali dlouhodobě. Práce se proto zaměřuje nejen na technické řešení jednotlivých funkcí, ale také na dokumentaci celého vývoje.

Jedním z hlavních cílů je vytvořit spolehlivou funkční část aplikace, která bude pracovat s databází a bezpečně ukládat informace o uživatelích a jejich úkolech. To zahrnuje návrh databázového modelu, jeho realizaci v MySQL a vytvoření potřebných mechanismů pro přihlašování a práci s daty. Důležitou součástí je také zabezpečení aplikace: správné ukládání hesel, ochrana osobních údajů a kontrola přístupu tak, aby k jednotlivým částem systému měli přístup jen přihlášení a oprávnění uživatelé. Bezpečnost je klíčová, protože se pracuje s osobními daty uživatelů.

Dalším cílem je vytvoření přehledného uživatelského rozhraní, které bude jednoduché na ovládání a zároveň funkční. Aplikace musí správně fungovat v běžných webových prohlížečích a měla by být použitelná na různých zařízeních. Zásadní částí rozhraní je správa úkolů — jejich přidávání, úprava, mazání a označení jako splněných. Úkoly jsou doplněny o práci s termíny, aby uživatel získal přehled o tom, kdy jednotlivé povinnosti končí a mohl si podle toho plánovat svůj čas. Aplikace tak propojuje jednoduchý seznam úkolů s kalendářovým pohledem, který pomáhá s orientací v nadcházejících termínech.

Nedílnou součástí cílů je i zapojení motivačních prvků založených na gamifikaci. Uživatelé získávají za splněné úkoly body, které mohou ovlivňovat jejich úroveň nebo další statistiky. Tyto funkce mají za úkol podpořit pravidelné používání aplikace a vytvořit u studentů pozitivní pracovní návyky. Aplikace tak neslouží jen k organizaci času, ale také k motivaci a sebereflexi uživatele.

Součástí finálního cíle je otestování aplikace a vyhodnocení její funkčnosti. Testování proběhne s několika uživateli, kteří aplikaci vyzkoušejí v praxi. Na základě jejich zpětné vazby budou popsány možné problémy, návrhy na vylepšení a celkové zhodnocení toho, zda aplikace splnila svůj účel. Do práce tak bude zahrnuta nejen technická stránka, ale i praktické ověření a zhodnocení její použitelnosti.

Cílem maturitní práce tedy není pouze vytvořit jednoduchý seznam úkolů, ale komplexnější nástroj spojující plánování, kalendář a motivační prvky. Výsledná aplikace by měla studentům nejen pomoci s přehledem o jejich povinnostech, ale také podpořit rozvoj osobní organizace a disciplíny.

1 ANALÝZA PRÁCE

1.1 Analýza zadání

Zadáním práce bylo vytvořit webovou aplikaci, která umožní efektivní správu úkolů, plánování času a organizaci studijních povinností. Základní myšlenkou projektu je vytvoření systému, který propojuje evidenci úkolů, kalendář a motivační prvky do jednoho přehledného prostředí.

Prvním hlavním požadavkem je vytvoření systému pro správu úkolů. Uživatel s oprávněním učitele nebo administrátora musí mít možnost vytvářet nové úkoly, zadávat jejich název, popis a termín splnění a ukládat tyto údaje do databáze. Součástí správy je také možnost úkoly upravovat, označovat jako splněné nebo je odstraňovat. Důležitou funkcí je také možnost přiřazení úkolů konkrétním uživatelům, vybraným skupinám nebo všem uživatelům.

Dalším důležitým požadavkem je vytvoření uživatelského rozhraní, které umožní přehledné zobrazení úkolů. Aplikace musí poskytovat seznam úkolů a zároveň jejich zobrazení v kalendáři podle termínu splnění. Kalendářová část systému slouží k lepší orientaci v čase a plánování jednotlivých aktivit.

Součástí zadání je také práce s uživatelskými účty a rolemi. Systém rozlišuje administrátora, učitele a studenta. Každá z těchto rolí má odlišná oprávnění. Administrátor má přístup ke správě uživatelů, učitel může vytvářet a spravovat úkoly a student má přístup pouze ke svým přiřazeným úkolům.

Dalším požadavkem je implementace motivačních prvků. Systém musí umožňovat přidělování bodů za splněné úkoly a jejich ukládání do databáze. Body jsou následně využity pro vytvoření žebříčku uživatelů, který podporuje aktivitu a pravidelné používání aplikace.

Z technického hlediska je důležitým požadavkem využití databáze pro ukládání všech dat. Databáze obsahuje informace o uživatelích, úkolech, bodech a dalších částech systému. Zajišťuje také propojení jednotlivých funkcí aplikace.

Minimálním cílem projektu bylo vytvořit funkční webovou aplikaci, která umožní správu úkolů, práci s uživatelskými účty a zobrazení dat v přehledné formě. Rozšířením tohoto řešení je implementace kalendáře, bodového systému, žebříčku uživatelů a dalších funkcí, které zvyšují použitelnost aplikace.

1.2 Cílová skupina

Cílovou skupinou aplikace jsou studenti, kteří potřebují efektivně spravovat své školní povinnosti a plánovat čas. Studium je spojeno s velkým množstvím úkolů, projektů a testů, jejichž přehledné evidování bývá problematické. Často nejde o nedostatek snahy, ale o absenci jednotného nástroje, který by umožnil mít všechny povinnosti na jednom místě.

Studenti často využívají různé aplikace nebo papírové poznámky, které však neumožňují propojení úkolů s kalendářem ani dlouhodobé sledování plnění. Výsledkem je ztráta přehledu, zapomínání termínů a horší organizace času.

Z analýzy vyplývá potřeba jednoduchého a přehledného systému, který umožní evidenci úkolů, práci s termíny a jejich zobrazení v kalendáři. Důležitým požadavkem je také dostupnost na různých zařízeních a intuitivní ovládání bez nutnosti složitého nastavování. Součástí systému by měly být i motivační prvky, které podpoří pravidelné plnění úkolů.

1.3 Analýza konkurence

Na trhu existuje několik systémů, které se zaměřují na správu školních povinností, organizaci výuky nebo komunikaci mezi studenty a učiteli. Mezi nejznámější patří například Moodle, Bakaláři nebo různé obecné plánovací aplikace.

Moodle je jeden z nejrozšířenějších LMS systémů, tedy systémů pro řízení výuky. Slouží především k tvorbě a správě online kurzů, sdílení studijních materiálů, zadávání úkolů a testování studentů . Umožňuje také sledování studijních výsledků a komunikaci mezi studenty a učiteli. Jeho výhodou je široká funkcionalita a možnost přizpůsobení, nevýhodou je však vyšší složitost a menší přehlednost pro běžné každodenní plánování.

Dalším často používaným systémem v českém školním prostředí jsou Bakaláři. Tento systém slouží především jako školní informační systém, který eviduje známky, docházku, rozvrh a základní komunikaci mezi školou a studenty. Výhodou je jeho rozšířenost a napojení na školní administrativu. Nevýhodou je omezená podpora plánování a správy vlastních úkolů z pohledu studenta.

Vedle těchto specializovaných systémů existují také obecné aplikace pro správu úkolů, například Google Tasks, Todoist nebo Trello. Tyto nástroje umožňují vytváření úkolů a plánování času, ale nejsou přizpůsobeny školnímu prostředí. Chybí zde například práce s rolemi, propojení se školními aktivitami nebo motivační prvky.

Aplikace Achievo se od těchto řešení liší především svým zaměřením. Nejde o komplexní školní systém ani o obecný plánovač, ale o nástroj zaměřený přímo na studenty. Kombinuje správu úkolů, kalendář a motivační systém bodů v jednom prostředí. Díky tomu poskytuje jednodušší a přehlednější řešení pro každodenní organizaci studia.

2 NÁVRH PRÁCE

2.1 Návrh databáze

Datový model aplikace je navržen jako relační databáze, která umožňuje efektivní ukládání a správu dat. Pro realizaci byla zvolena databáze MySQL, která poskytuje spolehlivé ukládání dat a podporu vztahů mezi tabulkami.

2.1.1 Tabulka users

Slouží k ukládání informací o registrovaných uživatelích aplikace. Obsahuje základní údaje potřebné pro přihlášení, identifikaci uživatele a správu uživatelských účtů.

Mezi ukládané údaje patří uživatelské jméno, e-mailová adresa, hashované heslo, datum vytvoření účtu, role uživatele a případně také obrázek rozvrhu. Role uživatele určuje jeho oprávnění v systému. Uživatel může být administrátor, učitel nebo student.

2.1.2 Tabulka events

Slouží k ukládání událostí spojených s konkrétním uživatelem. Tato tabulka umožňuje evidovat různé školní nebo osobní aktivity, které se následně zobrazují v kalendářové části aplikace.

Každý záznam obsahuje identifikátor uživatele, název události, popis, čas začátku, čas ukončení a typ události. Typ může určovat, zda jde například o test, projekt, hodinu nebo jinou aktivitu.

2.1.3 Tabulka tasks

Obsahuje hlavní informace o jednotlivých úkolech zadávaných v aplikaci. Každý záznam představuje jeden konkrétní úkol.

Ukládají se zde údaje jako název úkolu, popis, termín splnění, stav úkolu, datum vytvoření, typ přiřazení, seznam přiřazených uživatelů, maximální počet bodů a informace o archivaci. Součástí záznamu je také identifikátor uživatele, který úkol vytvořil.

2.1.4 Tabulka points

Slouží k ukládání bodového hodnocení za splněné úkoly. Tato tabulka propojuje uživatele, úkoly a osobu, která body přidělila.

Každý záznam obsahuje identifikátor uživatele, identifikátor úkolu, počet přidělených bodů, identifikátor uživatele, který body udělil, poznámku a čas vytvoření nebo úpravy záznamu. Díky tomu je možné sledovat, kdo body získal, za jaký úkol byly přiděleny a kdo je udělil.

2.1.5 Vztahy mezi tabulkami

Jednotlivé tabulky databáze jsou mezi sebou propojeny pomocí cizích klíčů, které zajišťují logickou návaznost dat. Tyto vztahy umožňují efektivní ukládání i načítání informací v celé aplikaci.

Mezi tabulkami users a events je vztah typu 1:N, protože jeden uživatel může mít více událostí. Tabulka events obsahuje identifikátor user_id, který odkazuje na tabulku users.

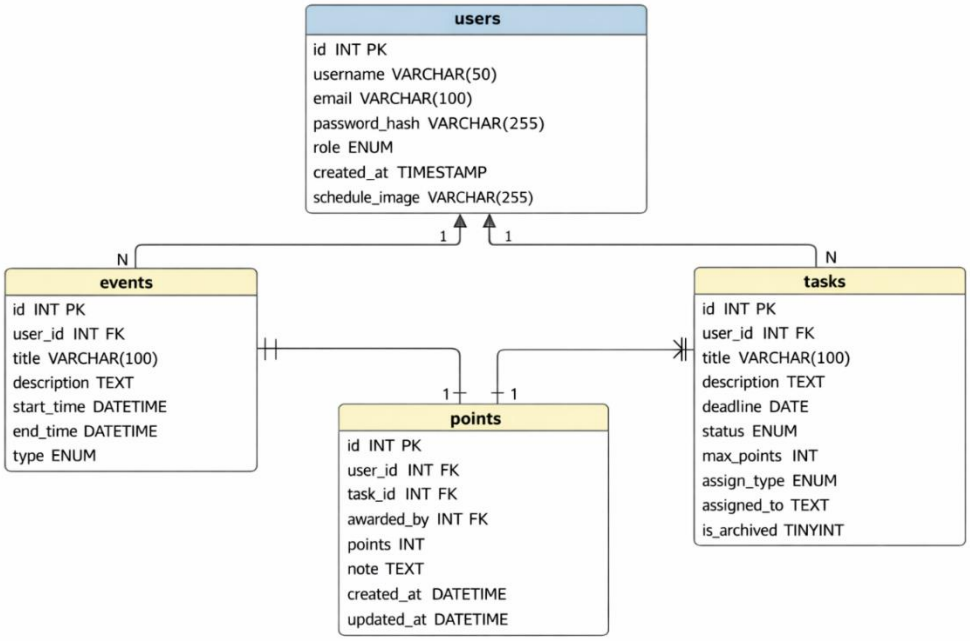
Podobný vztah typu 1:N je také mezi tabulkami users a tasks, protože jeden uživatel může vytvořit více úkolů. Tabulka tasks obsahuje identifikátor user_id, který určuje autora úkolu.

Další vztah typu 1:N je mezi tabulkami tasks a points, protože jeden úkol může mít více bodových záznamů. Tabulka points obsahuje identifikátor task_id, který odkazuje na tabulku tasks.

Mezi tabulkami users a points existuje také vztah typu 1:N, protože jeden uživatel může získat více bodových záznamů. Tento vztah je realizován pomocí atributu user_id v tabulce points.

Současně je mezi tabulkami users a points ještě další vztah typu 1:N prostřednictvím atributu awarded_by, protože jeden uživatel, například učitel nebo administrátor, může přidělit body více uživatelům.

Databázový model je navržen tak, aby podporoval správu uživatelů, úkolů, událostí a bodového systému. Vztahy mezi tabulkami zároveň zajišťují konzistenci dat a umožňují další rozšíření aplikace o nové funkce.



Obrázek 1 : ER diagram databáze

2.2 Návrh architektury systému

Aplikace Achievo je navržena jako webová aplikace využívající architekturu MVC, která rozděluje systém na tři základní části: model, view a controller.

Tento přístup zajišťuje přehlednost kódu, oddělení jednotlivých částí aplikace a jednodušší údržbu.

2.2.1 Celková struktura systému

Systém je tvořen třemi hlavními vrstvami:

• prezentační vrstva (frontend)
• aplikační vrstva (backend)
• datová vrstva (databáze)

Frontend zajišťuje zobrazení uživatelského rozhraní a interakci s uživatelem. Backend zpracovává požadavky, provádí aplikační logiku a komunikuje s databází. Databáze slouží k ukládání všech dat potřebných pro chod aplikace.

Jednotlivé vrstvy spolu komunikují prostřednictvím HTTP požadavků a odpovědí, což umožňuje oddělení jednotlivých částí systému a jejich nezávislý vývoj.

2.2.2 Frontend

Frontend představuje část aplikace, se kterou uživatel přímo pracuje. Je tvořen pomocí HTML, CSS a JavaScriptu.

Zajišťuje zobrazení jednotlivých stránek, formulářů a interaktivních prvků. Uživatel prostřednictvím frontendové části zadává vstupní data, například při vytváření úkolu nebo přihlášení.

Frontend odesílá požadavky na backend, který je následně zpracovává.

2.2.3 Backend

Backend je implementován v jazyce PHP s využitím frameworku CodeIgniter 4.

Tato část aplikace zajišťuje zpracování požadavků, práci s databází a aplikační logiku.

Každý požadavek od uživatele je směrován na konkrétní controller, který provede potřebné operace.

Controller komunikuje s modelem, který zajišťuje práci s databází.

Backend také zajišťuje:

- autentizaci uživatelů
- kontrolu oprávnění
- validaci vstupních dat
- zpracování formulářů

Výsledkem zpracování je odpověď, která je vrácena frontendové části.

2.2.4 Databázová vrstva

Databázová vrstva je realizována pomocí systému MySQL.

Slouží k ukládání všech dat aplikace, včetně uživatelských účtů, úkolů a bodů.

Komunikace s databází probíhá pomocí SQL dotazů. Tyto dotazy jsou vytvářeny v modelové vrstvě aplikace.

Pro zvýšení bezpečnosti jsou využívány parametrizované dotazy, které minimalizují riziko SQL injection.

2.2.5 Komunikace mezi vrstvami

Komunikace mezi frontendem a backendem probíhá pomocí HTTP požadavků.

Uživatel provede akci ve frontendové části, například odešle formulář. Tento požadavek je odeslán na server, kde jej zpracuje backend.

Backend následně načte nebo upraví data v databázi a vrátí odpověď. Data mohou být předávána například ve formátu JSON.

Frontend na základě odpovědi aktualizuje zobrazení stránky.

2.2.6 Řízení přístupu

Přístup k jednotlivým částem aplikace je řízen pomocí autentizace a uživatelských rolí.

Po přihlášení jsou informace o uživateli uloženy do session. Každý požadavek je následně kontrolován, aby bylo ověřeno, zda má uživatel oprávnění k dané akci.

Například administrátorské funkce jsou dostupné pouze uživatelům s odpovídající rolí.

Tento mechanismus zajišťuje bezpečnost a ochranu dat v aplikaci.

2.3 Návrh uživatelského rozhraní

Při návrhu uživatelského rozhraní aplikace Achievo byl kladen důraz především na jednoduchost, přehlednost a snadné ovládání. Cílovou skupinou jsou studenti, kteří potřebují rychlý přístup ke svým úkolům, termínům a přehledu o plnění povinností. Uživatelské rozhraní proto musí být srozumitelné i pro uživatele bez technických znalostí a zároveň dostatečně přehledné pro každodenní používání.

Aplikace je navržena jako webová, takže musí správně fungovat v běžných internetových prohlížečích a být použitelná na počítači i mobilním zařízení. Rozhraní je proto řešeno responzivně, aby se jednotlivé prvky přizpůsobovaly velikosti obrazovky. Důraz byl kladen na logické rozvržení jednotlivých částí systému a na to, aby se uživatel v aplikaci mohl rychle orientovat.

Navigace je řešena pomocí horní navigační lišty, která obsahuje odkazy na hlavní části aplikace. Mezi tyto části patří Dashboard, Úkoly, Kalendář, Rozvrh, Leaderboard a v případě administrátora také Administrace. Díky tomuto řešení má uživatel neustále přístup k nejdůležitějším sekcím systému.

Hlavní stránkou aplikace je Dashboard, který slouží jako přehledové centrum. Uživatel zde vidí základní informace o svých úkolech, počtu nesplněných povinností, získaných bodech a nejbližších termínech. Smyslem této stránky je poskytnout rychlý přehled bez nutnosti přecházet do dalších částí aplikace.

Sekce Úkoly je navržena jako přehledný seznam, ve kterém jsou jednotlivé úkoly řazeny podle termínu splnění. Každý úkol obsahuje základní informace, například název, popis, termín a stav splnění. Důležité je také vizuální odlišení splněných a nesplněných úkolů, které zlepšuje orientaci v seznamu. Uživatelé s odpovídajícím oprávněním mohou úkoly vytvářet, upravovat nebo archivovat.

Kalendářová část aplikace doplňuje seznam úkolů o časový pohled. Uživatel zde vidí své povinnosti rozložené podle dnů, což mu pomáhá při plánování času. Rozhraní kalendáře musí být přehledné, aby bylo možné rychle rozpoznat blížící se termíny a důležité události. Kliknutím na konkrétní den lze zobrazit detail záznamů.

Součástí návrhu je také leaderboard, který zobrazuje uživatele podle počtu získaných bodů. Tato část má motivační charakter a podporuje aktivní používání aplikace. Rozhraní leaderboardu je navrženo jednoduše, aby bylo možné snadno porovnat výsledky jednotlivých uživatelů.

Administrátorská část aplikace je oddělena od běžného uživatelského rozhraní a je dostupná pouze uživatelům s příslušnou rolí. Slouží ke správě uživatelských účtů a dalších systémových funkcí. Toto oddělení přispívá k větší bezpečnosti i přehlednosti aplikace.

Celkově je uživatelské rozhraní navrženo tak, aby podporovalo rychlou orientaci, jednoduchou práci s úkoly a přehledné zobrazení důležitých informací. Cílem návrhu je vytvořit prostředí, které bude pro studenty praktické, srozumitelné a vhodné pro každodenní používání.

2.4 Bezpečnostní návrh

Při návrhu bezpečnosti aplikace Achievo byl kladen důraz na ochranu uživatelských účtů, osobních údajů a omezení přístupu k citlivým částem systému. Jelikož aplikace pracuje s daty uživatelů, úkoly a bodovým hodnocením, je nutné zajistit, aby k těmto informacím měli přístup pouze oprávnění uživatelé.

Základním bezpečnostním prvkem je autentizace uživatelů. Přístup do chráněných částí aplikace je umožněn pouze po zadání správného uživatelského jména a hesla. Hesla nejsou v databázi ukládána v čitelné podobě, ale ve formě hashů vytvořených funkcí password_hash() s využitím algoritmu bcrypt. Při přihlášení dochází k ověření hesla pomocí funkce password_verify(). Tento postup výrazně snižuje riziko zneužití údajů při případném úniku databáze.

Dalším důležitým prvkem je práce se session. Po úspěšném přihlášení jsou do session uloženy základní informace o uživateli, například jeho identifikátor, role a stav přihlášení. Při každém požadavku na chráněnou část systému se ověřuje, zda session existuje a zda obsahuje platné údaje. Pokud tomu tak není, je uživatel přesměrován na přihlašovací stránku.

Významnou součástí bezpečnostního návrhu je také řízení přístupu podle uživatelských rolí. Systém rozlišuje běžného uživatele, učitele a administrátora. Každá role má odlišná oprávnění. Běžný uživatel může pracovat se svými úkoly a zobrazovat dostupné informace. Učitel nebo administrátor má navíc přístup k vytváření a správě úkolů. Administrátor může spravovat také uživatelské účty. Kontrola oprávnění probíhá na backendu, aby nebylo možné pravidla obejít pouze úpravou uživatelského rozhraní.

Pro ochranu databáze před neoprávněnými zásahy jsou při komunikaci s databází využívány parametrizované dotazy. Tento postup omezuje riziko SQL injection, tedy útoku založeného na vložení škodlivého SQL kódu do vstupních polí formulářů.

Součástí návrhu je také validace vstupních dat. Při odesílání formulářů jsou kontrolovány povinné hodnoty, datové typy a rozsah vstupů. Tím se snižuje riziko ukládání neplatných nebo škodlivých dat do systému. Validace probíhá jak na straně klienta pro lepší uživatelský komfort, tak na straně serveru pro skutečné zajištění bezpečnosti.

Důležitým opatřením je i omezení přístupu k administrátorské části aplikace. Tato část není dostupná běžným uživatelům a přístup k ní je podmíněn kontrolou role v session. Tím je zajištěno, že správu uživatelů a další citlivé operace může provádět pouze oprávněná osoba.

Bezpečnostní návrh počítá také s bezpečným odhlášením uživatele. Při odhlášení dochází ke zničení session, čímž se zabrání dalšímu přístupu k chráněným částem aplikace bez nového přihlášení.

Celkově je bezpečnostní návrh zaměřen na ochranu identity uživatelů, zabezpečení uložených dat a omezení přístupu k funkcím podle oprávnění. Tato opatření tvoří základ pro bezpečný provoz webové aplikace a snižují riziko zneužití systému.

3 TECHNICKÉ ZPRACOVÁNÍ

3.1 Použité technologie

Pro zhotovení webové aplikace Achievo jsem zvolil technologie, umožňující vytvořit stabilní, bezpečný a přehledný systém. Vybrány byly s ohledem na rozsah projektu, dostupné znalosti a požadavky na provoz webové aplikace bez nutnosti instalace na straně uživatele.

3.1.1 Jazyk PHP

Serverovou logiku aplikace implementuje jazyk PHP, ten jsem zvolil vzhledem k rozsáhlosti a dynamice webové aplikace. Výhodou je také jednoduchá syntaxe a rozsáhlá dokumentace. Jazyk také obsahuje vestavěné funkce pro práci s hesly, př. password_hash() a password_verify(), které jsem sám využil pro bezpečné ukládání přihlašovacích údajů.
Alternativou PHP je např. Javascript s pomocí Node.js. Tyto technologie však mají složitější konfiguraci a pro rozsah mé maturitní práce byl PHP vhodnější hlavně díky jednoduchosti a funkčnosti.

3.1.2 Framework CodeIgniter4

Pro struktorování aplikace jsem použil framework CodeIgniter verze 4. Využívá architekturu MVC, tedy odděluje aplikační logiky, práce s daty a uživatelské rozhraní.
Alternativou byl framework Laravel. Ten nabízí širší funkcionalitu, př. ORM systém nebo vestavěné autentizační moduly. Nevýhodou je však vyšší náročnost na pochopení a daleko větší složitost projektu. Z těchto důvodů jsem zvolil Codeigniter jako effektivnější řešení pro malý obsah.

3.1.3 Databázoví systém MySQL

Pro ukládání dat byla vybrána relační databáze MySQL. Tento systém umožňuje práci s tabulkami, relacemi a s daty uživatelů. V databázi jsou uložená veškerá data. Správa databáze je prováděna pomocí nástroje phpMyAdmin.

4 IMPLEMENTACE DŮLEŽITÝCH FUNKCÍ

4.1 Autentizace uživatelů

Autentizace uživatelů slouží k ověření identity a řízení přístupu do jednotlivých částí aplikace. Tento mechanismus zajišťuje, že k chráněným funkcím mají přístup pouze registrovaní a přihlášení uživatelé.

Proces přihlášení začíná zadáním uživatelského jména a hesla do přihlašovacího formuláře. Po odeslání formuláře jsou data předána na server, kde probíhá jejich zpracování. Server nejprve ověří existenci uživatele v databázi a následně porovná zadané heslo s uloženým hashovaným heslem pomocí funkce password_verify().

Hesla nejsou ukládána v čitelné podobě, ale ve formě hashů vytvořených funkcí password_hash() s využitím algoritmu bcrypt. Tento přístup zvyšuje bezpečnost aplikace a minimalizuje riziko zneužití dat.

Po úspěšném přihlášení jsou do session uloženy základní informace o uživateli, například jeho identifikátor, role a stav přihlášení. Tyto údaje jsou následně využívány při dalších požadavcích pro ověření oprávnění uživatele.

V případě neúspěšného přihlášení je uživateli zobrazena chybová hláška a přístup do aplikace není umožněn. Součástí systému je také odhlášení, při kterém dochází ke zničení session a ztrátě přístupu k chráněným částem aplikace.

```php
public function processLogin()
{
    $session = session();
    $userModel = new UserModel();

    $username = $this->request->getPost('username');
    $password = $this->request->getPost('password');

    $user = $userModel->where('username', $username)->first();

    if (!$user) {
        return redirect()->back()->with('error', 'Uživatel neexistuje.');
    }

    if (!password_verify($password, $user['password_hash'])) {
        return redirect()->back()->with('error', 'Špatné heslo.');
    }

    $session->set([
        'user_id'   => $user['id'],
        'username'  => $user['username'],
        'role'      => $user['role'],
        'logged_in' => true
    ]);

    return redirect()->to('/dashboard');
}
```

Obrázek 2 : Ukázka uložení session uživatele

4.1.1 Správa session

Session slouží k uchování informací o přihlášeném uživateli mezi jednotlivými požadavky. Bez tohoto mechanismu by nebylo možné udržet stav přihlášení.

Po úspěšném přihlášení jsou do session uloženy údaje jako user_id, role a příznak přihlášení. Tyto hodnoty jsou kontrolovány při každém přístupu k chráněným částem aplikace.

Pokud session neexistuje nebo neobsahuje požadované údaje, je uživatel přesměrován na přihlašovací stránku. Session je spravována na straně serveru, což zvyšuje bezpečnost, protože citlivá data nejsou ukládána v klientském prostředí.

Při odhlášení dochází k odstranění všech dat ze session a jejímu zničení. Tím je zajištěno, že uživatel nemůže pokračovat v práci bez opětovného přihlášení.

```
$session->set([
    'user_id'   => $user['id'],
    'username'  => $user['username'],
    'role'      => $user['role'],
    'logged_in' => true
]);
```

Obrázek 3 : Ukázka získání dat o session uživatele

```
if (!session()->get('logged_in')) {
    return redirect()->to('/login');
}
```

Obrázek 4 : Ukázka kontroly přihlášené session

4.2 Správa úkolů

Správa úkolů představuje hlavní funkční část aplikace. Umožňuje vytváření, úpravu, mazání a vyhodnocování úkolů, které jsou ukládány v databázi.

Každý úkol je reprezentován záznamem v tabulce tasks. Tento záznam obsahuje název, popis, termín splnění, identifikátor autora a způsob přiřazení.

Při vytvoření úkolu uživatel vyplní formulář, jehož data jsou po odeslání validována a uložena do databáze pomocí SQL dotazu. Aplikace podporuje různé způsoby přiřazení úkolů, například jednotlivci, vybraným uživatelům nebo všem uživatelům.

Při načítání úkolů systém filtruje data podle přihlášeného uživatele a jeho role. Úkoly jsou řazeny podle termínu splnění, což umožňuje lepší orientaci v povinnostech.

```php
public function add()
{
    if (!session()->get('logged_in')) {
        return redirect()->to('/login');
    }

    if (!in_array(session()->get('role'), ['admin', 'teacher'], true)) {
        return redirect()->to('/tasks');
    }

    $userModel = new UserModel();

    $data['users'] = $userModel
        ->where('id !=', session()->get('user_id'))
        ->findAll();

    $data['title'] = 'Přidat úkol';

    return view('task_add', $data);
}
```

Obrázek 5 : Ukázka kontroly uživatele a vytvoření formuláře pro vytvoření úkolu

```php
public function store()
{
    if (!session()->get('logged_in')) {
        return redirect()->to('/login');
    }

    if (!in_array(session()->get('role'), ['admin', 'teacher'], true)) {
        return redirect()->to('/tasks');
    }

    $taskModel = new TaskModel();

    $assignType = $this->request->getPost('assign_type');
    $selectedUsers = $this->request->getPost('selected_users') ?? [];
    $maxPoints = (int) $this->request->getPost('max_points');

    if ($maxPoints < 1) {
        return redirect()->back()->withInput()->with('error', 'Maximální počet bodů musí být alespoň 1.');
    }

    $assignedTo = null;

    if ($assignType === 'single' && !empty($selectedUsers)) {
        $assignedTo = $selectedUsers[0];
    }

    if ($assignType === 'selected' && !empty($selectedUsers)) {
        $assignedTo = implode(',', $selectedUsers);
    }

    if ($assignType === 'all') {
        $assignedTo = null;
    }

    $taskModel->insert([
        'user_id' => session()->get('user_id'),
        'title' => $this->request->getPost('title'),
        'description' => $this->request->getPost('description'),
        'deadline' => $this->request->getPost('deadline'),
        'status' => 'pending',
        'created_at' => date('Y-m-d H:i:s'),
        'assign_type' => $assignType,
        'assigned_to' => $assignedTo,
        'max_points' => $maxPoints
    ]);

    return redirect()->to('/tasks')->with('success', 'Úkol byl vytvořen.');
}
```

Obrázek 6 : Ukázka vytvoření úkolu

4.2.1 Mazání úkolů

Mazání úkolu je řešeno buď odstraněním záznamu z databáze, nebo jeho označením jako archivovaný. Tento přístup umožňuje zachování historických dat při současném odstranění úkolu z aktivního seznamu.

4.2.2 Označení úkolu jako splněný

Uživatel může úkol označit jako splněný pomocí tlačítka v seznamu úkolů. Po této akci dojde k aktualizaci stavu úkolu v databázi.

Splněné úkoly jsou vizuálně odlišeny, například změnou barvy, což zlepšuje přehlednost a orientaci v seznamu úkolů.

```php
public function done($id)
{
    if (!session()->get('logged_in')) {
        return redirect()->to('/login');
    }

    $taskModel = new TaskModel();
    $task = $taskModel->find($id);

    if (!$task) {
        return redirect()->to('/tasks');
    }

    $role = session()->get('role');
    $currentUserId = (string) session()->get('user_id');
    $canAccess = false;

    if ($role === 'admin') {
        $canAccess = true;
    }

    if ((string) $task['user_id'] === $currentUserId) {
        $canAccess = true;
    }

    if ($task['assign_type'] === 'all') {
        $canAccess = true;
    }

    if ($task['assign_type'] === 'single' && (string) $task['assigned_to'] === $currentUserId) {
        $canAccess = true;
    }

    if ($task['assign_type'] === 'selected' && !empty($task['assigned_to'])) {
        $ids = array_map('trim', explode(',', $task['assigned_to']));
        if (in_array($currentUserId, $ids, true)) {
            $canAccess = true;
        }
    }

    if (!$canAccess) {
        return redirect()->to('/tasks');
    }

    $taskModel->update($id, ['status' => 'done']);

    return redirect()->to('/tasks')->with('success', 'Úkol byl dokončen.');
}
```

Obrázek 7 : Ukázka pro označení úkolu a ověření role uživatele

4.3 Kalendář

Kalendář slouží k přehlednému zobrazení úkolů a událostí v časovém kontextu. Uživatel tak získává lepší přehled o svých povinnostech.

Pro implementaci byl použit nástroj FullCalendar, který umožňuje dynamické vykreslování kalendáře ve webovém rozhraní. Data jsou načítána z databáze prostřednictvím backendu ve formátu JSON.

Každý úkol je převeden na kalendářovou událost podle hodnoty deadline. Uživatel může kliknutím na konkrétní den zobrazit detail úkolů a událostí.

```php
public function events()
{
    if (!session()->get('logged_in')) {
        return $this->response->setJSON([]);
    }

    $taskModel = new TaskModel();

    $userId = (string) session()->get('user_id');
    $role = session()->get('role');

    $allTasks = $taskModel->findAll();
    $visibleTasks = [];

    if ($role === 'admin') {
        $visibleTasks = $allTasks;
    } else {
        foreach ($allTasks as $task) {
            $isCreator = ((string) $task['user_id'] === $userId);

            if ($isCreator) {
                $visibleTasks[] = $task;
                continue;
            }

            if ($task['assign_type'] === 'all') {
                $visibleTasks[] = $task;
                continue;
            }

            if ($task['assign_type'] === 'single' && (string) $task['assigned_to'] === $userId) {
                $visibleTasks[] = $task;
                continue;
            }

            if ($task['assign_type'] === 'selected' && !empty($task['assigned_to'])) {
                $ids = array_map('trim', explode(',', $task['assigned_to']));
                if (in_array($userId, $ids, true)) {
                    $visibleTasks[] = $task;
                }
            }
        }
    }
```

Obrázek 8 : Ukázka funkce kalendáře a zobrazení úkolů v něm

```php
$events = [];

foreach ($visibleTasks as $task) {
    $color = '#dc3545';

    if ($task['status'] === 'done') {
        $color = '#198754';
    } elseif ($task['status'] === 'pending') {
        $color = '#fd7e14';
    }

    $events[] = [
        'title' => $task['title'],
        'start' => $task['deadline'],
        'color' => $color
    ];
}

return $this->response->setJSON($events);
```

Obrázek 9 : Ukázka zvýraznění stavu úkolu

4.4 Gamifikace

Gamifikace slouží k motivaci uživatele k pravidelnému používání aplikace. Je založena na bodovém systému.

Po splnění úkolu jsou uživateli automaticky přiděleny body, které jsou ukládány do databáze. Tyto body jsou následně využity pro zobrazení v leaderboardu, kde jsou uživatelé seřazeni podle dosažených výsledků.

Tento mechanismus podporuje aktivitu uživatelů a přidává do aplikace soutěžní prvek.

```php
$data = [
    'user_id' => $userId,
    'task_id' => $taskId,
    'points' => $points,
    'awarded_by' => session()->get('user_id'),
    'note' => $note,
];

if ($existing) {
    $pointsModel->update($existing['id'], $data);
} else {
    $pointsModel->insert($data);
}
```

Obrázek 10 : Kontrola uživatele a přidělení nebo upravení bodů

4.5 Administrace

Administrátorská část aplikace slouží ke správě uživatelských účtů a je dostupná pouze uživatelům s rolí administrátora.

Přístup k této části je řízen kontrolou role uložené v session. Pokud uživatel nemá potřebná oprávnění, je přístup zamítnut.

Administrátor může vytvářet nové uživatele, upravovat jejich údaje a zobrazovat seznam všech účtů. Při vytváření účtu je heslo automaticky hashováno před uložením do databáze.

Administrace tak umožňuje centrální správu uživatelů a zajišťuje kontrolu nad celým systémem.

```php
public function createUser()
{
    if (!session()->get('logged_in')) {
        return redirect()->to('/login');
    }

    if (session()->get('role') !== 'admin') {
        return redirect()->to('/dashboard');
    }

    $userModel = new UserModel();

    $username = $this->request->getPost('username');
    $email = $this->request->getPost('email');
    $password = $this->request->getPost('password');
    $role = $this->request->getPost('role');

    $userModel->insert([
        'username' => $username,
        'email' => $email,
        'password_hash' => password_hash($password, PASSWORD_DEFAULT),
        'role' => $role
    ]);

    return redirect()->to('/admin/users')->with('success', 'Uživatel byl vytvořen.');
}
```

Obrázek 11: Ukázka formuláře pro vytvoření uživatelů

4.6 Napojení na databázi

Napojení aplikace na databázi je realizováno pomocí databázového systému MySQL, který je spravován prostřednictvím nástroje phpMyAdmin. Databáze slouží jako centrální úložiště všech dat aplikace, včetně uživatelských účtů, úkolů, událostí a bodového systému.

Komunikace mezi backendem a databází probíhá pomocí SQL dotazů. Tyto dotazy slouží k ukládání, načítání, úpravě a mazání dat. Jsou využívány napříč celou aplikací, například při přihlášení uživatele, práci s úkoly nebo při přidělování bodů.

Databázové operace jsou realizovány v backendové části aplikace pomocí modelů, které zajišťují oddělení logiky aplikace od přímé práce s databází. Tento přístup zvyšuje přehlednost kódu a usnadňuje jeho údržbu.

Pro zvýšení bezpečnosti jsou při práci s databází využívány parametrizované dotazy, které minimalizují riziko SQL injection. Vstupní data jsou před zpracováním validována, aby nedocházelo k ukládání neplatných nebo škodlivých hodnot.

Databáze je navržena jako relační a jednotlivé tabulky jsou propojeny pomocí cizích klíčů, což zajišťuje konzistenci dat. Díky tomu je možné efektivně pracovat s vazbami mezi uživateli, úkoly, událostmi a body.

Nástroj phpMyAdmin slouží především pro správu databáze, kontrolu dat a případné manuální úpravy. Umožňuje také snadné vytváření tabulek, úpravu struktury databáze a provádění SQL dotazů během vývoje aplikace.

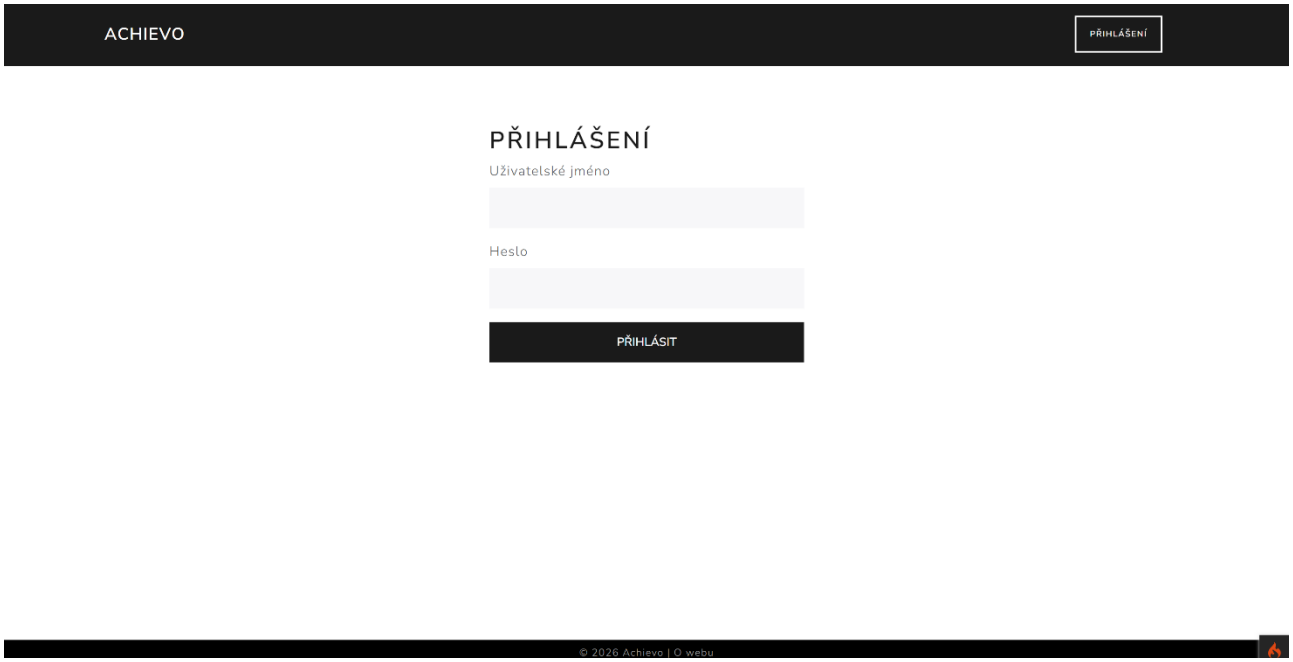
5 UŽIVATELSKÁ DOKUMENTACE

Tato kapitola slouží jako návod k používání webové aplikace Achievo. Popisuje jednotlivé kroky, které uživatel provádí při práci se systémem.

5.1 Přihlášení do aplikace

Uživatel otevře webovou aplikaci v internetovém prohlížeči. Na úvodní stránce se zobrazí přihlašovací formulář.

Do formuláře zadá uživatelské jméno a heslo a odešle jej pomocí tlačítka „Přihlásit“. Po úspěšném přihlášení je uživatel přesměrován na hlavní stránku aplikace.

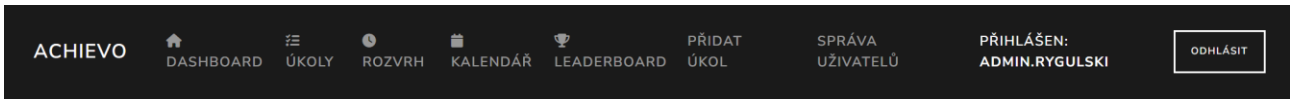


Obrázek 12 : Pohled na přihlašovací stránku

5.2 Orientace v aplikaci

Po přihlášení se zobrazí navigační lišta, která umožňuje přechod mezi jednotlivými částmi aplikace.

Mezi hlavní sekce patří Dashboard, Úkoly, Kalendář, Rozvrh a Leaderboard. Kliknutím na jednotlivé položky se uživatel přesouvá mezi stránkami.



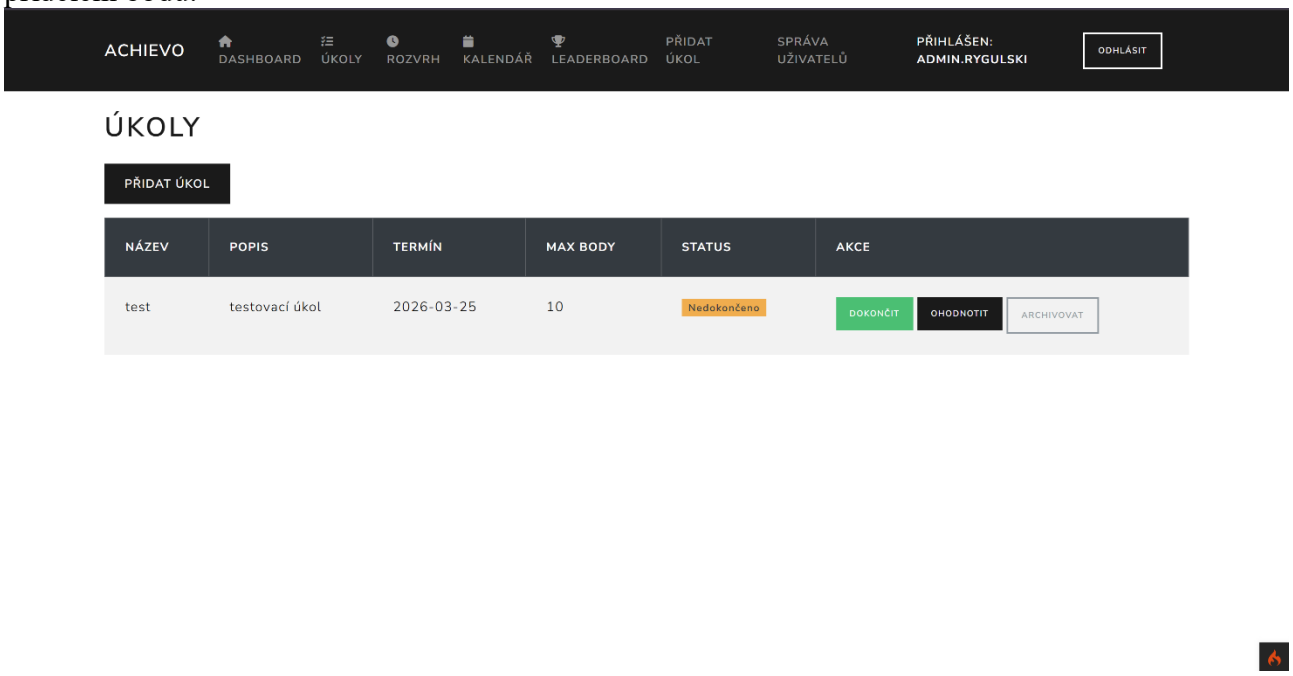
Obrázek 13 : Pohled na celý navbar

5.3 Práce s úkoly

Sekce úkolů slouží k evidenci a správě školních povinností.

Uživatel zde může zobrazit seznam úkolů, jejich termíny a aktuální stav. Úkoly jsou barevně rozlišeny podle toho, zda jsou splněné.

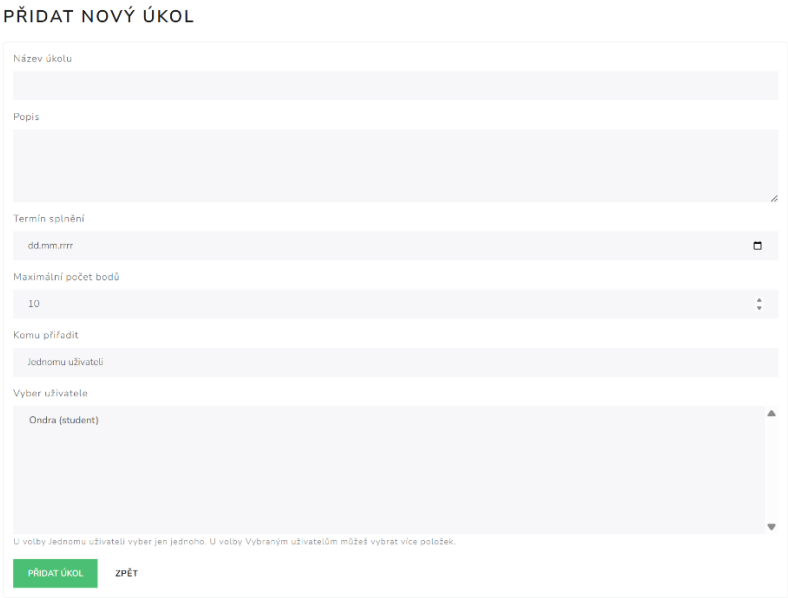
Uživatel může úkol označit jako splněný, čímž dojde k jeho aktualizaci v databázi a případnému přidělení bodů.



Obrázek 14 : Pohled na stránku s úkoly

Uživatel s oprávněním může vytvářet nové úkoly pomocí formuláře, kde vyplní název, popis a termín.

5.3.1 Vytvoření úkolu (učitel, admin)



Obrázek 15 : Formulář pro vytvoření úkolů

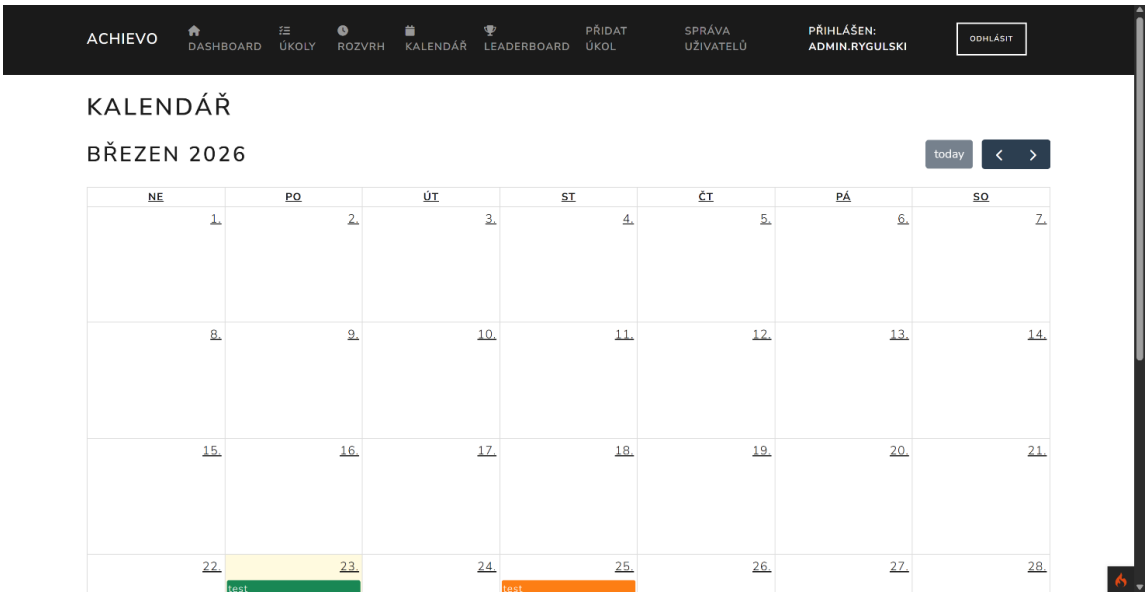
5.3.2 Hodnocení úkolů (učitel, admin)

Obrázek 16 : Formulář pro ohodnocení úkolů

5.4 Kalendář

Kalendář poskytuje přehled úkolů podle data. Po otevření stránky se zobrazí měsíční pohled.

Úkoly jsou automaticky načteny z databáze a zobrazeny v jednotlivých dnech. Kliknutím na konkrétní den lze zobrazit detail úkolů.



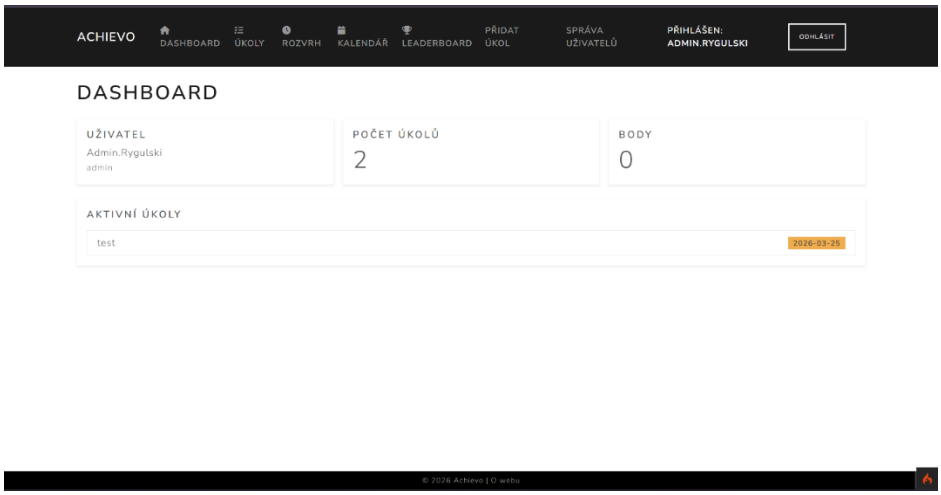
Obrázek 17 : Pohled na stránku s kalendářem

5.5 Dashboard

Dashboard slouží jako hlavní přehledová stránka.

Zobrazuje základní informace, jako je počet úkolů, počet nesplněných úkolů a aktuální počet bodů.

Součástí je také seznam nejbližších úkolů.

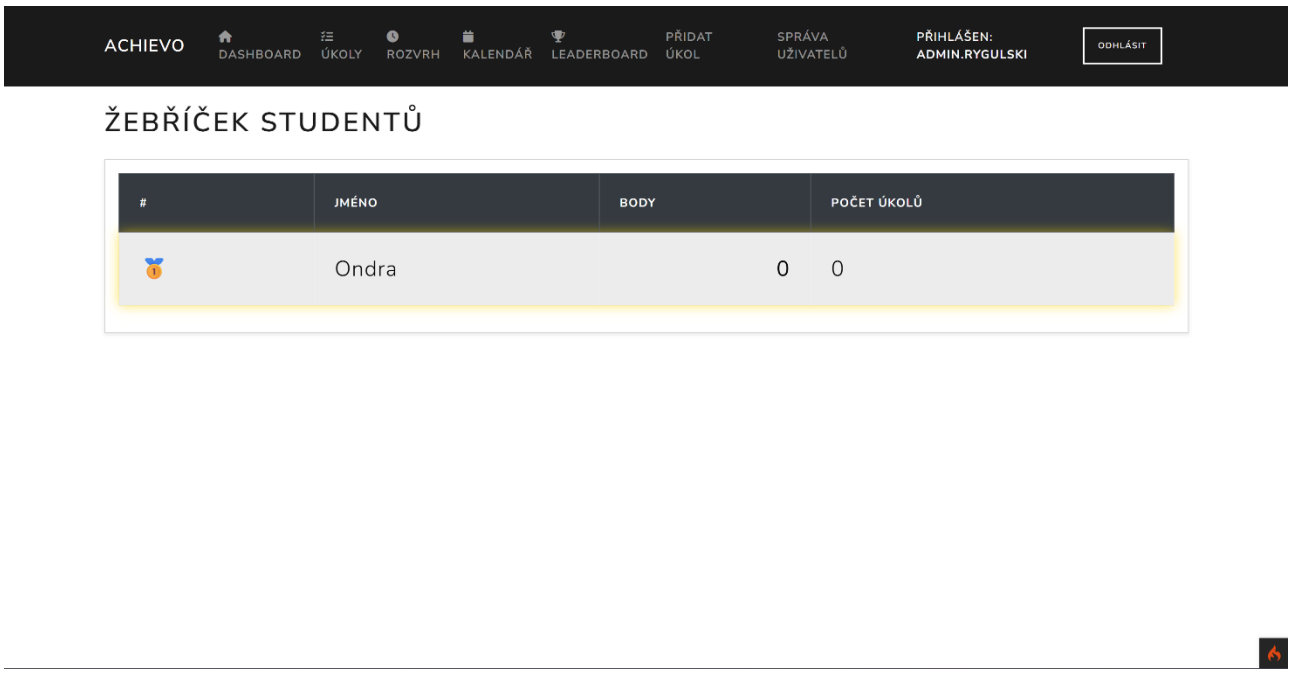


Obrázek 18 : Pohled na úvodní stránku Dashboard

5.6 Leaderboard

Leaderboard zobrazuje seznam uživatelů seřazených podle počtu bodů.

Tato stránka slouží k porovnání aktivity jednotlivých uživatelů.

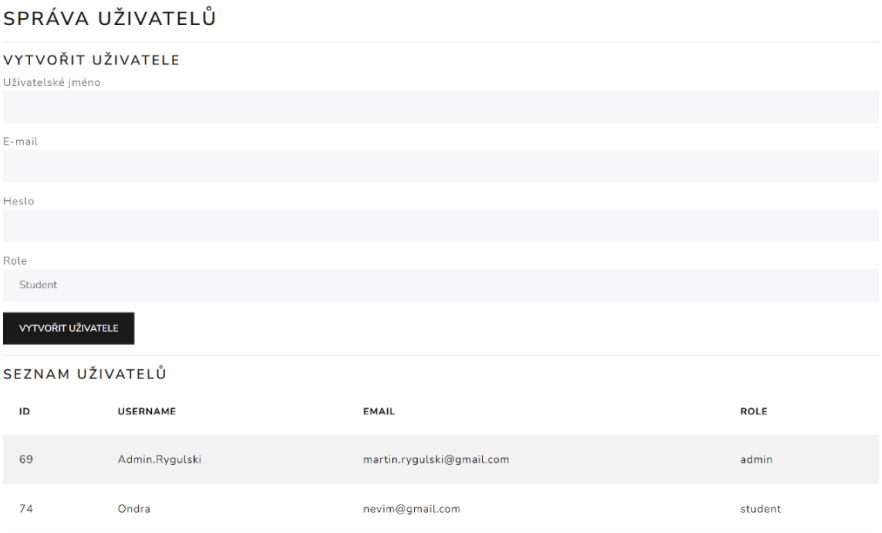


Obrázek 19: Pohled na stránku s "Žebříčkem studentů"

5.7 Administrace uživatelů (admin)

Administrátor má přístup ke správě uživatelů.

Může vytvářet nové účty, nastavovat role a zobrazovat seznam všech uživatelů.

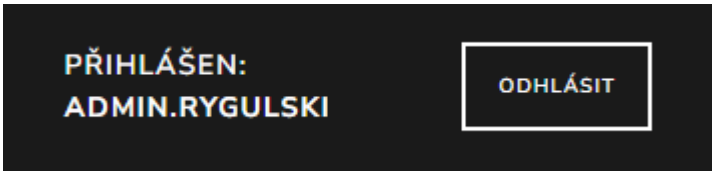


Obrázek 20 : Pohled na administrátorský formulář pro vytvoření a správu uživatelů

5.8 Odhlášení

Odhlášení je dostupné v horní části stránky.

Po kliknutí dojde k ukončení session a uživatel je přesměrován na přihlašovací stránku.



Obrázek 21 : Tlačítko odhlásit

6 UŽIVATELSKÉ TESTOVÁNÍ

Součástí vývoje webu bylo také uživatelské testování, jehož cílem bylo ověřit funkčnost, přehlednost a použitelnost jednotlivých částí systému. Testování proběhlo ve skupině uživatelů, kteří odpovídali cílové skupině aplikace.

6.1 Metodika testování

Testování probíhalo pro testery samostatně, bez žádného školení nebo pomoci. Bylo testováno zejména přihlašování, odhlašování, orientace na webu, funkčnost kalendáře i zobrazování, přidávání s odstraňování úkolů a tabulky.

6.2 Scénář testování

Každý tester dostal scénář postupu testování:

- Uživatel otevře webovou aplikaci v prohlížeči
- Uživatel se přihlásí pomocí svého uživatelského jména a hesla
- Uživatel se orientuje na hlavní stránce a přejde na další stránky
- Uživatel si vyuzkouší úkol zobrazit jak v Úkolech, tak Kalendáři
- Uživatel úkol odevzdá a získá body
- Uživatel přejde do Tabulky leaderů a uvidí se na ní
- Uživatel se odhlásí ze systému

6.3 Jednotlivé testování:

6.3.1 Tester 1:

Tester byl student střední školy, který odpovídá cílové skupině aplikace. Testování probíhalo na běžném notebooku v prostředí webového prohlížeče.
Tester nejprve provedl přihlášení do systému, které proběhlo bez problémů. Následně se orientoval na hlavní stránce a postupně prošel jednotlivé sekce aplikace, zejména Úkoly, Kalendář a Dashboard.
Při práci s úkoly tester ocenil přehlednost seznamu a jednoduché označení úkolu jako splněného. Pozitivně hodnotil také řazení úkolů podle termínu, které mu pomohlo rychle se zorientovat v prioritách.

V kalendáři tester vyzkoušel zobrazení úkolů podle jednotlivých dnů. Funkčnost označil jako bezproblémovou, ale navrhl zvýraznit aktuální den, aby byla orientace ještě rychlejší.
Celkově tester hodnotil aplikaci jako přehlednou a snadno použitelnou.

6.3.2 Tester 2:

Tester byl student se zaměřením na informační technologie. Testování probíhalo na stolním počítači v prostorách školy.

Tester se zaměřil především na funkčnost jednotlivých částí aplikace. Po přihlášení prošel všechny hlavní sekce a testoval práci s úkoly, kalendářem a leaderboardem.

Při vytváření úkolu tester ověřil správné uložení dat do databáze. Funkce fungovala bez chyb. Dále testoval mazání úkolů a jejich označení jako splněné, přičemž všechny operace proběhly korektně.

V oblasti kalendáře tester ocenil propojení s úkoly a správné zobrazení dat.

Celkově hodnotil aplikaci jako funkční a dobře navrženou. Jeho připomínky se týkaly především drobných vylepšení, např. forma upozornění na nadcházející termíny.

Tato připomínka bude vyřešena při dalším rozšířením aplikace.

6.3.3 Tester 3:

Tester byl běžný uživatel bez technického zaměření. Testování probíhalo na mobilním zařízení..
Tester provedl základní průchod aplikací od přihlášení až po práci s jednotlivými funkcemi. Zaměřil se především na srozumitelnost a jednoduchost ovládání.
Přihlášení do aplikace proběhlo bez problémů. Tester se následně orientoval v navigaci a postupně navštívil jednotlivé sekce.
Při práci s úkoly ocenil jednoduchost ovládání, zejména možnost rychle označit úkol jako splněný.
V kalendáři se dokázal orientovat bez obtíží.
Tester upozornil na menší nedostatky v zobrazení na mobilním zařízení, například méně přehledné rozložení některých prvků.
Celkově označil aplikaci jako jednoduchou a srozumitelnou. Navrhl drobné úpravy responzivity pro lepší zobrazení na menších obrazovkách.
Na jeho doporučení jsem reagoval upravením gridu UI aplikace, čímž jsem tento problém vyřešil.

6.3.4 Tester 4:

Tester byl student střední školy bez technického zaměření. Testování probíhalo na stolním počítači v běžném webovém prohlížeči.

Tester provedl kompletní průchod aplikací podle zadaného scénáře. Po přihlášení se zaměřil na orientaci v uživatelském rozhraní a přehlednost jednotlivých prvků.

Na hlavní stránce ocenil jednoduchý přehled informací a rychlý přístup k úkolům. V sekci úkolů tester bez problémů pochopil, jak úkol vytvořit i označit jako splněný.

Při práci s kalendářem tester uvedl, že by uvítal možnost zobrazit detail úkolu přímo po kliknutí bez nutnosti dalšího přechodu.

V leaderboardu tester ocenil jednoduchost a přehlednost zobrazení uživatelů.

Celkově tester hodnotil aplikaci jako intuitivní a snadno použitelnou.

6.3.5 Tester 5:

Tester byl student se základními znalostmi práce s počítačem. Testování probíhalo na notebooku a mobilním zařízení.

Tester se zaměřil především na použitelnost aplikace na menším zařízení. Po přihlášení prošel všechny hlavní sekce aplikace.

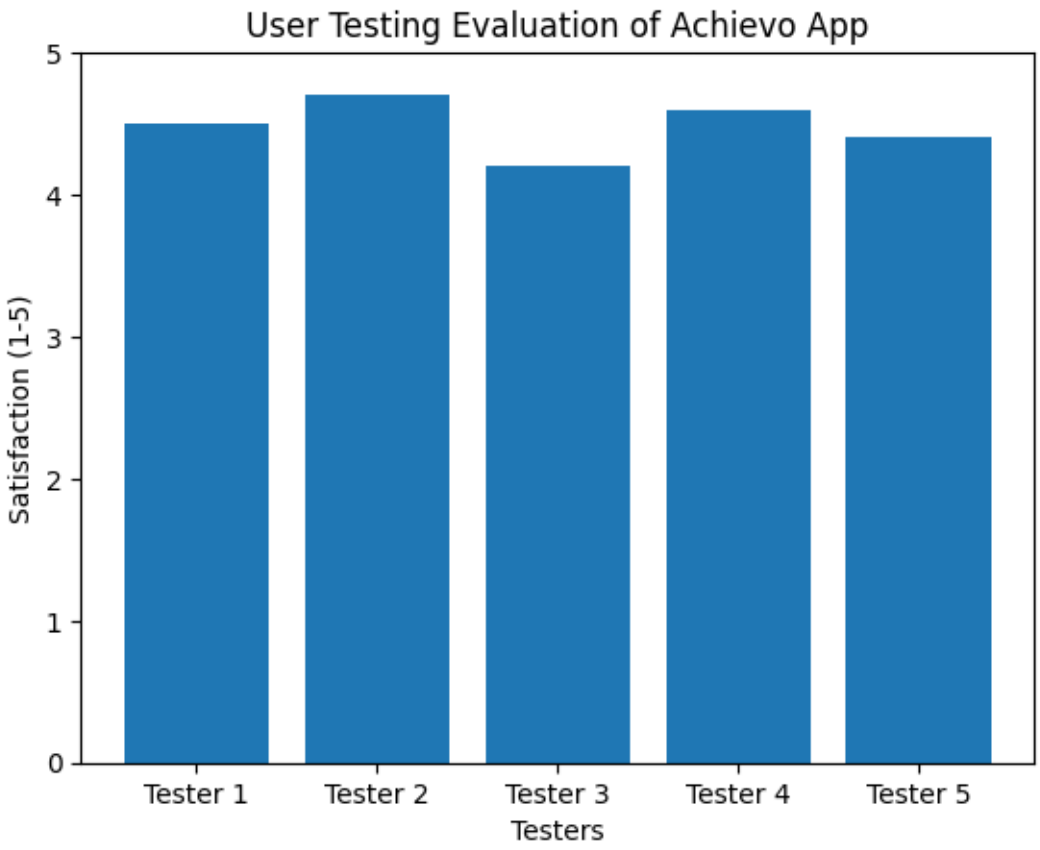
V sekci úkolů tester ocenil jednoduchost ovládání a rychlou reakci systému. Bez problémů vytvořil nový úkol a následně jej označil jako splněný.

Při práci s kalendářem tester uvedl, že zobrazení je funkční, ale na menší obrazovce by mohlo být přehlednější.

Tester také vyzkoušel leaderboard a dashboard, kde ocenil přehledné zobrazení informací.

Celkově hodnotil aplikaci pozitivně, zejména z hlediska jednoduchosti a funkčnosti.

6.4 Finální hodnocení pomocí formuláře



Obrázek 22: Graf z hodnocení testerů

Graf znázorňuje celkové hodnocení aplikace jednotlivými testery. Všichni testeři hodnotili aplikaci pozitivně, přičemž nejlépe byla hodnocena funkčnost a přehlednost systému. Nejnižší hodnocení souviselo s drobnými nedostatky v responzivitě a uživatelském rozhraní na menších zařízeních.

7 PŘÍNOSY PRÁCE

Hlavním přínosem této práce je návrh a realizace funkční webové aplikace, která řeší konkrétní problém studentů, a to organizaci školních povinností a plánování času. Aplikace Achievo poskytuje jednotné prostředí, ve kterém má uživatel přehled o úkolech, termínech i celkové aktivitě. Tím odstraňuje nutnost používat více různých nástrojů a zjednodušuje každodenní práci.

Významným přínosem je propojení správy úkolů s kalendářovým zobrazením. Uživatel nevidí pouze seznam povinností, ale také jejich časové rozložení. Díky tomu získává lepší přehled o svém rozvrhu a může efektivněji plánovat svůj čas. Tento přístup pomáhá předcházet zapomínání termínů a zlepšuje organizaci studia.

Dalším přínosem je implementace bodového systému, který přidává do aplikace motivační prvek. Uživatelé získávají body za splněné úkoly a mohou sledovat své výsledky v rámci leaderboardu. Tento mechanismus podporuje pravidelné plnění povinností a zvyšuje aktivitu uživatelů.

Z technického hlediska práce přináší kompletní návrh a realizaci webové aplikace založené na architektuře MVC. Systém je rozdělen na frontend, backend a databázovou vrstvu, což zajišťuje přehlednost kódu a usnadňuje jeho další rozvoj. Byla navržena relační databáze, která efektivně ukládá data o uživatelích, úkolech, událostech a bodech a umožňuje jejich vzájemné propojení.

Přínosem je také důraz na bezpečnost aplikace. Byla implementována autentizace uživatelů, práce se session a řízení přístupu podle rolí. Hesla jsou ukládána v hashované podobě a při komunikaci s databází jsou využívány parametrizované dotazy. Tato opatření zajišťují ochranu uživatelských dat a bezpečný provoz systému.

Z uživatelského hlediska je důležitá jednoduchost a přehlednost rozhraní. Aplikace je navržena tak, aby byla snadno ovladatelná i pro méně zkušené uživatele. Responzivní design umožňuje použití na různých zařízeních, což zvyšuje dostupnost aplikace.

Dalším přínosem je možnost rozšíření systému. Návrh databáze i architektury umožňuje přidání dalších funkcí, například notifikací, pokročilých statistik nebo dalších motivačních prvků. Aplikace tak může být dále rozvíjena podle potřeb uživatelů.

Práce má také praktický přínos pro autora, který získal zkušenosti s návrhem databází, vývojem webových aplikací a implementací bezpečnostních mechanismů. Získané znalosti je možné využít při dalším studiu nebo v praxi.

Celkově práce přináší ucelené řešení pro organizaci studijních povinností a ukazuje, jak lze pomocí moderních technologií vytvořit praktickou a využitelnou aplikaci.

ZÁVĚR

Cílem maturitní práce bylo navrhnout a vytvořit přehlednou webovou aplikaci Achievo určenou studentům pro správu času a plánování školních povinností. Tento cíl byl splněn. Byla vytvořena funkční aplikace umožňující evidenci úkolů, práci s termíny a jejich zobrazení v kalendářovém přehledu.

Aplikace byla implementována v jazyce PHP s využitím frameworku CodeIgniter a relační databáze MySQL. Byla navržena databázová struktura, realizována autentizace uživatelů se zabezpečeným ukládáním hesel pomocí algoritmu bcrypt a vytvořeno přehledné uživatelské rozhraní. Součástí systému je také jednoduchý motivační mechanismus, založený na přidělování bodů za splněné úkoly a umísťování v tabulce nejlepších.

Přínosem práce je vytvoření uceleného systému, který propojuje organizací úkolů, časové plánování a motivačními prvky do jednoho prostředí. Aplikace může studentům pomoci získat lepší přehled o jejich povinnostech, efektivněji si rozvrhnout práci a snížit riziko zapomenutí důležitých termínů. Zároveň práce přinesla praktické zkušenosti s návrhem databázového modelu, implementací architektury MVC a řešením základních bezpečnostních aspektů webových aplikací. Výsledný projektu představuje funkční základ webové aplikace, který lze dále rozšiřovat o další moduly a pokročilejší funkce.

SEZNAM POUŽITÉ LITERATURY

(1) W3SCHOOLS. *PHP for begginers*. Online. C1999-2025. Dostupné z: https://www.w3schools.com/php/. [cit. 2025-12-14].

(2) CODEIGNITER FOUNDATION. *Codeingiter 4*. Online. C2025. Dostupné z: https://www.codeigniter.com/user_guide/index.html. [cit. 2025-12-14].

(3) BOOTSWATCH. *Bootswatch: Flatty*. Online. C2014. Dostupné z: https://bootswatch.com/flatly/. [cit. 2025-12-14].

(4) W3SCHOOLS. Node.js Tutorial. Online. C1999-2026 Dostupné z: https://www.w3schools.com/nodejs/. [cit. 2026-03-17].

(5) BOOTSWATCH. Bootswatch Themes. Online. C2012–2025. Dostupné z: https://bootswatch.com/. [cit. 2025-12-14].

(6) FULLCALENDAR. FullCalendar Documentation. Online. C2010–2025. Dostupné z: https://fullcalendar.io/docs. [cit. 2025-12-14].

(7) WEBZDARMA. Webzdarma.cz. Online. Dostupné z: https://www.webzdarma.cz/. [cit. 2026-03-21].

(8) W3SCHOOLS. MySQL Tutorial. Online. C1999-2025. Dostupné z: https://www.w3schools.com/mysql/. [cit. 2025-12-14].

(9) W3SCHOOLS. HTML Tutorial. Online. C1999-2025. Dostupné z: https://www.w3schools.com/html/. [cit. 2025-12-14].

(10) W3SCHOOLS. CSS Tutorial. Online. C1999-2025. Dostupné z: https://www.w3schools.com/css/. [cit. 2025-12-14].

(11) PHP.NET. password_hash. Online. 2025. Dostupné z: https://www.php.net/manual/en/function.password-hash.php. [cit. 2025-12-14].

(12) PHP.NET. password_verify. Online. 2025. Dostupné z: https://www.php.net/manual/en/function.password-verify.php. [cit. 2025-12-14].

(13) MYSQL. MySQL Documentation. Online. 2025. Dostupné z: https://dev.mysql.com/doc/. [cit. 2025-12-14].

(14) BOOTSTRAP. Bootstrap Documentation. Online. 2025. Dostupné z: https://getbootstrap.com/docs/. [cit. 2025-12-14].

(15) TRAVERSY MEDIA. PHP For Beginners Crash Course. Online. 2023. Dostupné z: https://www.youtube.com/watch?v=OK_JCtrrv-c. [cit. 2025-12-14].

(16) PROGRAM WITH GIO. PHP Full Course for Beginners. Online. 2023. Dostupné z: https://www.youtube.com/watch?v=zZ6vybT1HQs. [cit. 2025-12-14].

(17) CODEIGNITER. CodeIgniter 4 Tutorial for Beginners. Online. 2022. Dostupné z: https://www.youtube.com/watch?v=R4cJzq6a0oQ. [cit. 2025-12-14].

(18) NET NINJA. MySQL Tutorial for Beginners. Online. 2022. Dostupné z: https://www.youtube.com/watch?v=7S_tz1z_5bA. [cit. 2025-12-14].

(19) NET NINJA. HTML & CSS Crash Course. Online. 2021. Dostupné z: https://www.youtube.com/watch?v=mU6anWqZJcc. [cit. 2025-12-14].

(20) FULLCALENDAR. FullCalendar Tutorial. Online. 2023. Dostupné z: https://www.youtube.com/watch?v=8M9t1t5Z3IY. [cit. 2025-12-14].

(21) PROGRAMMING WITH MOSH. JavaScript Tutorial for Beginners. Online. 2021. Dostupné z: https://www.youtube.com/watch?v=W6NZfCO5SIk. [cit. 2025-12-14].

(22) DANI KROSSING. PHP Tutorial for Beginners. Online. 2022. Dostupné z: https://www.youtube.com/watch?v=2eebptXfEvw. [cit. 2025-12-14].

(23) (CODE WITH HARRY. PHP Tutorial in One Video. Online. 2023. Dostupné z: https://www.youtube.com/watch?v=1SnPKhCdlsU. [cit. 2025-12-14].

(24) NET NINJA. JavaScript Tutorial for Beginners. Online. 2022. Dostupné z: https://www.youtube.com/watch?v=qoSksQ4s_hg. [cit. 2025-12-14].

SEZNAM POUŽITÝCH SYMBOLŮ A ZKRATEK

MVC – Model–View–Controller, architektura oddělující logiku aplikace, práci s daty a uživatelské rozhraní

PHP – serverový skriptovací jazyk použitý pro implementaci aplikační logiky

SQL – Structured Query Language, jazyk pro práci s databází

MySQL – relační databázový systém použitý pro ukládání dat

phpMyAdmin – nástroj pro správu databáze MySQL

CodeIgniter 4 – PHP framework využívající architekturu MVC

HTML – HyperText Markup Language, jazyk pro tvorbu struktury webových stránek

CSS – Cascading Style Sheets, jazyk pro stylování webového rozhraní

JSON – JavaScript Object Notation, formát pro přenos dat mezi serverem a klientem

FullCalendar – JavaScriptová knihovna pro zobrazení kalendáře

CRUD – základní operace nad daty (Create, Read, Update, Delete)

Session – mechanismus pro uchování dat o přihlášeném uživateli

ID – jednoznačný identifikátor záznamu v databázi

UI – User Interface, uživatelské rozhraní aplikace

UX – User Experience, uživatelský zážitek

bcrypt – algoritmus pro hashování hesel

API – rozhraní pro komunikaci mezi částmi systému

SEZNAM OBRÁZKŮ

Obrázek 1 : ER diagram databáze 15
Obrázek 2 : Ukázka uložení session uživatele 20
Obrázek 3 : Ukázka získání dat o session uživatele 21
Obrázek 4 : Ukázka kontroly přihlášené session 21
Obrázek 5 : Ukázka kontroly uživatele a vytvoření formuláře pro vytvoření úkolu 22
Obrázek 6 : Ukázka vytvoření úkolu 22
Obrázek 7 : Ukázka pro označení úkolu a ověření role uživatele 23
Obrázek 8 : Ukázka funkce kalendáře a zobrazení úkolů v něm 24
Obrázek 9 : Ukázka zvýraznění stavu úkolu 25
Obrázek 10 : Kontrola uživatele a přidělení nebo upravení bodů 25
Obrázek 11: Ukázka formuláře pro vytvoření uživatelů 26
Obrázek 12 : Pohled na přihlašovací stránku 28
Obrázek 13 : Pohled na celý navbar 29
Obrázek 14 : Pohled na stránku s úkoly 29
Obrázek 15 : Formulář pro vytvoření úkolů 30
Obrázek 16 : Formulář pro ohodnocení úkolů 30
Obrázek 17 : Pohled na stránku s kalendářem 31
Obrázek 18 : Pohled na úvodní stránku Dashboard 31
Obrázek 19: Pohled na stránku s "Žebříčkem studentů" 32
Obrázek 20 : Pohled na administrátorský formulář pro vytvoření a správu uživatelů 32
Obrázek 21 : Tlačítko odhlásit 33
Obrázek 22: Graf z hodnocení testerů 37